16. 키 관리 및 키스토어 구현 분석

개요

Lighthouse에서 키 관리는 Ethereum 2.0 검증자의 핵심 보안 요소입니다. 이 문서는 `eth2_keystore` 와 `eth2_key_derivation` 모듈의 구현을 상세히 분석하여, BLS 키의 생성, 저장, 로딩 과정을 실제 코드와 함께 설명합니다.

키 관리 시스템은 다음과 같은 주요 컴포넌트로 구성됩니다:

- **키 파생 (Key Derivation)**: EIP-2333 표준에 따른 계층적 키 생성
- **키스토어 (Keystore)**: EIP-2335 표준에 따른 암호화된 키 저장
- **지갑 (Wallet)**: 다중 검증자 키 관리를 위한 상위 레벨 추상화

키 파생 시스템 (eth2_key_derivation)

EIP-2333 표준 구현

Lighthouse는 [EIP-2333](#) 표준을 구현하여 BLS12-381 키의 계층적 파생을 수행합니다.

DerivedKey 구조체

```
// crypto/eth2_key_derivation/src/derived_key.rs

#[derive(Zeroize)]

#[zeroize(drop)]

pub struct DerivedKey(ZeroizeHash);
```

```rust
impl DerivedKey {
/// 시드로부터 마스터 키 생성
pub fn from_seed(seed: &[u8]) -> Result<Self, Error> {
if seed.is_empty() {
Err(Error::EmptySeed)
} else {
Ok(Self(derive_master_sk(seed)))
}
}

/// 자식 키 파생
pub fn child(&self, index: u32) -> DerivedKey {
Self(derive_child_sk(self.0.as_bytes(), index))
}

/// BLS 비밀키 반환
pub fn secret(&self) -> &[u8] {
self.0.as_bytes()
}
}
```

마스터 키 파생 과정

마스터 키 생성은 `derive_master_sk` 함수를 통해 수행됩니다:

```
/// EIP-2333의 derive_master_SK와 동등
fn derive_master_sk(seed: &[u8]) -> ZeroizeHash {
hkdf_mod_r(seed)
}
```

HKDF-mod-r 알고리즘

핵심 키 파생 함수인 `hkdf_mod_r` 는 다음과 같이 구현됩니다:

```
fn hkdf_mod_r(ikm: &[u8]) -> ZeroizeHash {
// ikm = ikm + I2OSP(0,1)
let mut ikm_with_postfix = SecretBytes::zero(ikm.len() + 1);
ikm_with_postfix.as_mut_bytes()[..ikm.len()].copy_from_slice(ikm);

// info = "" + I2OSP(L, 2)
let info = u16::try_from(MOD_R_L)
.expect("MOD_R_L too large")
.to_be_bytes();

let mut output = ZeroizeHash::zero();
let zero_hash = ZeroizeHash::zero();

let mut salt = b"BLS-SIG-KEYGEN-SALT-".to_vec();
while output.as_bytes() == zero_hash.as_bytes() {
```

```
let mut hasher = Sha256::new();
hasher.update(salt.as_slice());
salt = hasher.finalize().to_vec();

let prk = hkdf_extract(&salt, ikm_with_postfix.as_bytes());
let okm = &hkdf_expand(prk, &info, MOD_R_L);

output = mod_r(okm.as_bytes());
}
output
}
```

자식 키 파생 과정

자식 키는 부모 키와 인덱스를 사용하여 생성됩니다:

```
/// EIP-2333의 derive_child_SK와 동등
fn derive_child_sk(parent_sk: &[u8], index: u32) -> ZeroizeHash {
let compressed_lamport_pk = parent_sk_to_lamport_pk(parent_sk, index);
hkdf_mod_r(compressed_lamport_pk.as_bytes())
}
```

Lamport 서명 기반 파생

자식 키 파생은 Lamport 서명을 중간 단계로 사용합니다:

```
fn parent_sk_to_lamport_pk(ikm: &[u8], index: u32) -> ZeroizeHash {
let salt = index.to_be_bytes();
let not_ikm = flip_bits(ikm);


let lamports = [
ikm_to_lamport_sk(&salt, ikm),
ikm_to_lamport_sk(&salt, not_ikm.as_bytes()),
];


// Lamport 공개키 생성 및 압축
let mut lamport_pk = SecretBytes::zero(HASH_SIZE * LAMPORT_ARRAY_SIZE as usize
let pk_bytes = lamport_pk.as_mut_bytes();


lamports
.iter()
.flat_map(LamportSecretKey::iter_chunks)
.enumerate()
.for_each(|(i, chunk)| {
let mut hasher = Sha256::new();
hasher.update(chunk);
pk_bytes[i * HASH_SIZE..(i + 1) * HASH_SIZE].copy_from_slice(&hasher.finalize())
});
```

```
// 압축된 Lamport 공개키 생성
let mut compressed_lamport_pk = ZeroizeHash::zero();
let mut hasher = Sha256::new();
hasher.update(lamport_pk.as_bytes());
compressed_lamport_pk
.as_mut_bytes()
.copy_from_slice(&hasher.finalize());


compressed_lamport_pk
}
```

키스토어 시스템 (eth2_keystore)

EIP-2335 표준 구현

키스토어는 [EIP-2335](#) 표준을 따라 BLS 키를 안전하게 저장합니다.

Keystore 구조체

```
// crypto/eth2_keystore/src/keystore.rs
#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
#[serde(transparent)]
pub struct Keystore {
json: JsonKeystore,
```

```
}


impl Keystore {
/// 키페어와 패스워드로부터 키스토어 생성
fn  encrypt(
keypair: &Keypair,
password: &[u8],
kdf: Kdf,
cipher: Cipher,
uuid: Uuid,
path: String,
description: String,
) -> Result<Self, Error> {
let  secret: ZeroizeHash = keypair.sk.serialize();
let (cipher_text, checksum) = encrypt(secret.as_bytes(), password, &kdf, &cipher


Ok(Keystore {
json: JsonKeystore {
crypto: Crypto {
kdf: KdfModule {
function: kdf.function(),
params: kdf,
message: EmptyString,
```

```
},
checksum: ChecksumModule {
function: Sha256Checksum::function(),
params: EmptyMap,
message: checksum.to_vec().into(),
},
cipher: CipherModule {
function: cipher.function(),
params: cipher,
message: cipher_text.into(),
},
},
uuid,
path: Some(path),
pubkey: keypair.pk.as_hex_string()[2..].to_string(),
version: Version::four(),
description: Some(description),
name: None,
},
})
}

/// 패스워드로 키페어 복호화
```

```
pub fn decrypt_keypair(&self, password: &[u8]) -> Result<Keypair, Error> {
let plain_text = decrypt(password, &self.json.crypto)?;

if plain_text.len() != SECRET_KEY_LEN {
return Err(Error::InvalidSecretKeyLen {
len: plain_text.len(),
expected: SECRET_KEY_LEN,
});
}

let keypair = keypair_from_secret(plain_text.as_bytes())?;
// 공개키 검증
if keypair.pk.as_hex_string()[2..] != self.json.pubkey {
return Err(Error::PublicKeyMismatch);
}

Ok(keypair)
}
}
```

KeystoreBuilder 패턴

키스토어 생성은 빌더 패턴을 사용합니다:

```
pub struct KeystoreBuilder<'a> {
keypair: &'a Keypair,
password: &'a [u8],
kdf: Kdf,
cipher: Cipher,
uuid: Uuid,
path: String,
description: String,
}

impl<'a> KeystoreBuilder<'a> {
/// 새 빌더 생성 (KDF salt와 AES IV를 랜덤 생성)
pub fn new(keypair: &'a Keypair, password: &'a [u8], path: String) -> Result<S
if password.is_empty() {
Err(Error::EmptyPassword)
} else {
let salt = rand::thread_rng().gen::<[u8; SALT_SIZE]>();
let iv = rand::thread_rng().gen::<[u8; IV_SIZE]>().to_vec().into();

Ok(Self {
keypair,
password,
```

```
kdf: default_kdf(salt.to_vec()),

cipher: Cipher::Aes128Ctr(Aes128Ctr { iv }),

uuid: Uuid::new_v4(),

path,

description: "".to_string(),

})

}

}


/// 키스토어 빌드

pub fn build(self) -> Result<Keystore, Error> {

Keystore::encrypt(

self.keypair,

self.password,

self.kdf,

self.cipher,

self.uuid,

self.path,

self.description,

)

}

}
```

암호화 및 복호화 과정

암호화 함수

```
pub fn encrypt(
plain_text: &[u8],
password: &[u8],
kdf: &Kdf,
cipher: &Cipher,
) -> Result<(Vec<u8>, [u8; HASH_SIZE]), Error> {
validate_parameters(kdf)?;
let mut password = normalize(password)?;

// EIP-2335에 따른 패스워드 정규화
password.retain(|c| !is_control_character(c));

let derived_key = derive_key(password.as_ref(), kdf)?;

// AES-128-CTR로 암호화
let mut cipher_text = plain_text.to_vec();
match &cipher {
Cipher::Aes128Ctr(params) => {
validate_aes_iv(params.iv.as_bytes())?;
```

```
let key = GenericArray::from_slice(&derived_key.as_bytes()[0..16]);
let nonce = GenericArray::from_slice(params.iv.as_bytes());
let mut cipher = AesCtr::new(key, nonce);
cipher.apply_keystream(&mut cipher_text);
}
};

let checksum = generate_checksum(&derived_key, &cipher_text);

Ok((cipher_text, checksum))
}
```

복호화 함수

```
pub fn decrypt(password: &[u8], crypto: &Crypto) -> Result<PlainText, Error> {
let mut password = normalize(password)?;
password.retain(|c| !is_control_character(c));

validate_parameters(&crypto.kdf.params)?;

let cipher_message = &crypto.cipher.message;
let derived_key = derive_key(password.as_ref(), &crypto.kdf.params)?;

// 체크섬 검증 (잘못된 패스워드 감지)
```

```
if &generate_checksum(&derived_key, cipher_message.as_bytes())[..]
!= crypto.checksum.message.as_bytes()
{
return Err(Error::InvalidPassword);
}

let mut plain_text = PlainText::from(cipher_message.as_bytes().to_vec());
match &crypto.cipher.params {
Cipher::Aes128Ctr(params) => {
validate_aes_iv(params.iv.as_bytes())?;

let key = GenericArray::from_slice(&derived_key.as_bytes()[0..16]);
let nonce = GenericArray::from_slice(params.iv.as_bytes());
let mut cipher = AesCtr::new(key, nonce);
cipher.apply_keystream(plain_text.as_mut_bytes());
}
};
Ok(plain_text)
}
```

키 파생 함수 (KDF)

Lighthouse는 두 가지 KDF를 지원합니다:

Scrypt (기본값)

```
pub fn default_kdf(salt: Vec<u8>) -> Kdf {
Kdf::Scrypt(Scrypt::default_scrypt(salt))
}

// Scrypt 파라미터 검증
fn validate_parameters(kdf: &Kdf) -> Result<(), Error> {
match kdf {
Kdf::Scrypt(params) => {
if params.n <= 1 || params.r == 0 || params.p == 0 {
return Err(Error::InvalidScryptParam);
}

if params.dklen != DKLEN {
return Err(Error::InvalidScryptParam);
}

// n이 2의 거듭제곱인지 확인
if params.n != 2u32.pow(log2_int(params.n)) {
return Err(Error::InvalidScryptParam);
}
```

```
// 메모리 사용량 계산: 128*n*p*r
let mut npr: u32 = params
.n
.checked_mul(params.p)
.ok_or(Error::InvalidScryptParam)?;
npr = npr.checked_mul(params.r).ok_or(Error::InvalidScryptParam)?;
npr = npr.checked_mul(128).ok_or(Error::InvalidScryptParam)?;

Ok(())
}
// PBKDF2 검증 로직...
}
}
```

PBKDF2

```
fn derive_key(password: &[u8], kdf: &Kdf) -> Result<DerivedKey, Error> {
let mut dk = DerivedKey::zero();

match &kdf {
Kdf::Pbkdf2(params) => {
pbkdf2::<Hmac<Sha256>>(
password,
params.salt.as_bytes(),
```

```
params.c,
dk.as_mut_bytes(),
);
}
Kdf::Scrypt(params) => {
scrypt(
password,
params.salt.as_bytes(),
&ScryptParams::new(log2_int(params.n) as u8, params.r, params.p)
.map_err(Error::ScryptInvalidParams)?,
dk.as_mut_bytes(),
)
.map_err(Error::ScryptInvaidOutputLen)?;
}
}

Ok(dk)
}
```

지갑 시스템 (eth2_wallet)

계층적 결정론적 지갑

지갑은 단일 시드로부터 여러 검증자 키를 생성할 수 있습니다:

```
// crypto/eth2_wallet/src/wallet.rs

#[derive(Debug, PartialEq, Serialize, Deserialize)]

#[serde(transparent)]

pub struct Wallet {

json: JsonWallet,

}



impl Wallet {

/// 다음 검증자 키스토어 생성

pub fn next_validator(

&mut self,

wallet_password: &[u8],

voting_keystore_password: &[u8],

withdrawal_keystore_password: &[u8],

) -> Result<ValidatorKeystores, Error> {

let derive = |key_type: KeyType, password: &[u8]| -> Result<Keystore, Error> {

let (secret, path) =

recover_validator_secret(self, wallet_password, self.json.nextaccount, key_type)



let keypair = keypair_from_secret(secret.as_bytes())?;
```

```
KeystoreBuilder::new(&keypair, password, format!("{}", path))?
.build()
.map_err(Into::into)
};

let  keystores = ValidatorKeystores {
voting: derive(KeyType::Voting, voting_keystore_password)?,
withdrawal: derive(KeyType::Withdrawal, withdrawal_keystore_password)?,
};

self.json.nextaccount = self
.json
.nextaccount
.checked_add(1)
.ok_or(Error::PathExhausted)?;

Ok(keystores)
}
}
```

ValidatorPath 구조

검증자 키는 표준화된 경로를 따릅니다:

```rust
// crypto/eth2_wallet/src/validator_path.rs

pub const PURPOSE: u32 = 12381;

pub const COIN_TYPE: u32 = 3600;


pub struct ValidatorPath {

account: u32,

key_type: KeyType,

}


impl ValidatorPath {

pub fn new(account: u32, key_type: KeyType) -> Self {

Self { account, key_type }

}


/// BIP-44 경로 노드들 반환: m/12381/3600/account/withdrawal_or_voting

pub fn iter_nodes(&self) -> impl Iterator<Item = &u32> {

[PURPOSE, COIN_TYPE, self.account, self.key_type as u32].iter()

}

}


#[derive(Debug, Clone, Copy, PartialEq)]

pub enum KeyType {
```

```
Voting = 0,
Withdrawal = 1,
}
```

BIP-39 니모닉 지원

지갑은 BIP-39 니모닉으로부터 생성할 수 있습니다:

```
impl<'a> WalletBuilder<'a> {
/// BIP-39 니모닉으로부터 지갑 빌더 생성
pub fn from_mnemonic(
mnemonic: &Mnemonic,
password: &'a [u8],
name: String,
) -> Result<Self, Error> {
let seed = Bip39Seed::new(mnemonic, "");
Self::from_seed_bytes(seed.as_bytes(), password, name)
}

/// 시드 바이트로부터 지갑 빌더 생성
pub fn from_seed_bytes(seed: &[u8], password: &'a [u8], name: String) -> Resul
if password.is_empty() {
Err(Error::EmptyPassword)
} else if seed.is_empty() {
```

```
Err(Error::EmptySeed)

} else {

let  salt = rand::thread_rng().gen::<[u8; SALT_SIZE]>();

let  iv = rand::thread_rng().gen::<[u8; IV_SIZE]>().to_vec().into();



Ok(Self {

seed: seed.to_vec().into(),

password,

kdf: default_kdf(salt.to_vec()),

cipher: Cipher::Aes128Ctr(Aes128Ctr { iv }),

uuid: Uuid::new_v4(),

nextaccount: 0,

name,

})

}

}

}
```

Validator Client에서의 사용

InitializedValidators에서의 키스토어 관리

```
// validator_client/initialized_validators/src/lib.rs
```

```
impl InitializedValidators {
/// 키스토어 파일로부터 검증자 로딩
pub fn from_keystore_paths(
keystore_paths: Vec<PathBuf>,
password_dir: &Path,
secrets_dir: &Path,
) -> Result<Self, Error> {
let mut validators = HashMap::new();
for keystore_path in keystore_paths {
let keystore = Keystore::from_json_file(&keystore_path)
.map_err(Error::UnableToParseVotingKeystore)?;
let password = load_password(&keystore, password_dir, secrets_dir)?;
let keypair = keystore
.decrypt_keypair(password.as_ref())
.map_err(Error::UnableToDecryptKeystore)?;
validators.insert(keypair.pk.clone(), keypair);
}
Ok(Self { validators })
}
}
```

서명 방법 (SigningMethod)

```rust
// validator_client/signing_method/src/lib.rs

pub enum SigningMethod {

/// 로컬 키스토어 사용

LocalKeystore {

voting_keystore: Keystore,

voting_keystore_lockfile: Mutex<Option<Lockfile>>,

voting_keystore_password: ZeroizeString,

},

/// 원격 서명자 사용 (Web3Signer)

Web3Signer {

signing_url: SensitiveUrl,

http_client: HttpClient,

voting_public_key: PublicKey,

},

}


impl SigningMethod {

/// BLS 서명 생성

pub async fn sign_message(

&self,

message: &[u8],

signing_root: Hash256,
```

```
) -> Result<Signature, Error> {
match  self {
SigningMethod::LocalKeystore {
voting_keystore,
voting_keystore_password,
..
} => {
let  keypair = voting_keystore
.decrypt_keypair(voting_keystore_password.as_ref())
.map_err(Error::UnableToDecryptKeystore)?;
Ok(keypair.sk.sign(message))
}
SigningMethod::Web3Signer { .. } => {
// Web3Signer를 통한 원격 서명
self.web3signer_sign_message(message, signing_root).await
}
}
}
}
```

보안 고려사항

메모리 보안

모든 민감한 데이터는 `Zeroize` 트레이트를 구현하여 메모리에서 안전하게 제거됩니다:

```
#[derive(Zeroize)]
#[zeroize(drop)]
pub struct DerivedKey(ZeroizeHash);

#[derive(Zeroize, Clone, PartialEq)]
#[zeroize(drop)]
pub struct PlainText(Vec<u8>);
```

패스워드 정규화

EIP-2335에 따라 패스워드는 NFKD 정규화되고 제어 문자가 제거됩니다:

```
fn normalize(bytes: &[u8]) -> Result<Zeroizing<String>, Error> {
Ok(str::from_utf8(bytes)
.map_err(|_| Error::InvalidPasswordBytes)?
.nfkd()
.collect::<String>()
.into())
}

fn is_control_character(c: char) -> bool {
c.is_control()
```

```
}
```

파라미터 검증

KDF 파라미터는 보안성과 DoS 공격 방지를 위해 엄격히 검증됩니다:

```
// PBKDF2의 경우 최소 262,144회 반복, 최대 80,000,000회
if params.c < DEFAULT_PBKDF2_C {
if params.c == 0 {
return Err(Error::InvalidPbkdf2Param);
}
eprintln!("WARN: PBKDF2 parameters are too weak");
}

if params.c > 80_000_000 {
return Err(Error::InvalidPbkdf2Param);
}
```

실제 사용 예시

키스토어 생성 및 사용

```
use bls::Keypair;
use eth2_keystore::{KeystoreBuilder, Keystore};
```

```
// 1. 새 키페어 생성
let keypair = Keypair::random();
let password = b"secure_password";

// 2. 키스토어 생성
let keystore = KeystoreBuilder::new(&keypair, password, "m/12381/3600/0/0".to_s
.description("My validator key".to_string())
.build()?;

// 3. JSON으로 저장
let json_string = keystore.to_json_string()?;
std::fs::write("validator_keystore.json", json_string)?;

// 4. 키스토어 로딩 및 복호화
let loaded_keystore = Keystore::from_json_file("validator_keystore.json")?;
let recovered_keypair = loaded_keystore.decrypt_keypair(password)?;

assert_eq!(keypair.pk, recovered_keypair.pk);
```

지갑을 통한 다중 키 생성

```
use eth2_wallet::{WalletBuilder, bip39::Mnemonic};
```

```
// 1. 니모닉으로부터 지갑 생성
let  mnemonic = Mnemonic::parse("abandon abandon abandon abandon abandon abandon
let  wallet_password = b"wallet_password";


let  mut  wallet = WalletBuilder::from_mnemonic(&mnemonic, wallet_password, "My
.build()?;


// 2. 첫 번째 검증자 키 생성
let  voting_password = b"voting_password";
let  withdrawal_password = b"withdrawal_password";


let  validator_keystores = wallet.next_validator(
wallet_password,
voting_password,
withdrawal_password,
)?;


// 3. 키스토어 저장
validator_keystores.voting.to_json_writer(
std::fs::File::create("validator_0_voting.json")?
)?;
validator_keystores.withdrawal.to_json_writer(
std::fs::File::create("validator_0_withdrawal.json")?
```

```
)?;
```

결론

Lighthouse의 키 관리 시스템은 Ethereum 2.0 표준을 완전히 준수하면서도 강력한 보안성을 제공합니다. EIP-2333과 EIP-2335 표준의 구현을 통해 상호 운용성을 보장하고, Rust의 타입 시스템과 `Zeroize` 트레이트를 활용하여 메모리 안전성을 확보합니다.

주요 특징:

- **표준 준수**: EIP-2333, EIP-2335 완전 구현
- **계층적 키 파생**: 단일 시드로부터 무제한 키 생성
- **강력한 암호화**: Scrypt/PBKDF2 + AES-128-CTR
- **메모리 안전성**: 민감한 데이터의 안전한 제거
- **상호 운용성**: 다른 Ethereum 2.0 클라이언트와 호환

이러한 설계를 통해 Lighthouse는 검증자의 키를 안전하게 관리하면서도 사용자 친화적인 인터페이스를 제공합니다.